UNIVERSITY OF SCIENCE, VNUHCM

FACULTY OF INFORMATION TECHNOLOGY

SOFTWARE TESTING

HW05 – Performance Testing

1. Student Information & General Information

- Name: Trần Trí Nhân
- Student ID: 23127097
- Homework version: 2026.HW05.Performance Testing_En_2.0_HTThanh.pdf

Task 1

GitHub link for test plans GitHub link for test data

Workflow Selection

POST /api/login -> GET /api/cart -> POST /api/apply-coupon -> POST /api/checkout

Why this workflow:

- Login API covers auth-heavy group
- Get cart API covers read-heavy group
- Apply coupon and checkout APIs cover transactional group

Test plans review

Overall, the plans designed by the AI were reasonable and scoped correctly to the given workflow. The proposed settings for Thread Groups like the number of VUs/threads, ramp-up duration, timers, etc,... were realistic and suitable for specified performance testing scenarios. However, there were some parts or sections of the plan that I found unnecessary or incorrect, so I adjusted these to make the plans more accuracy and simpler.

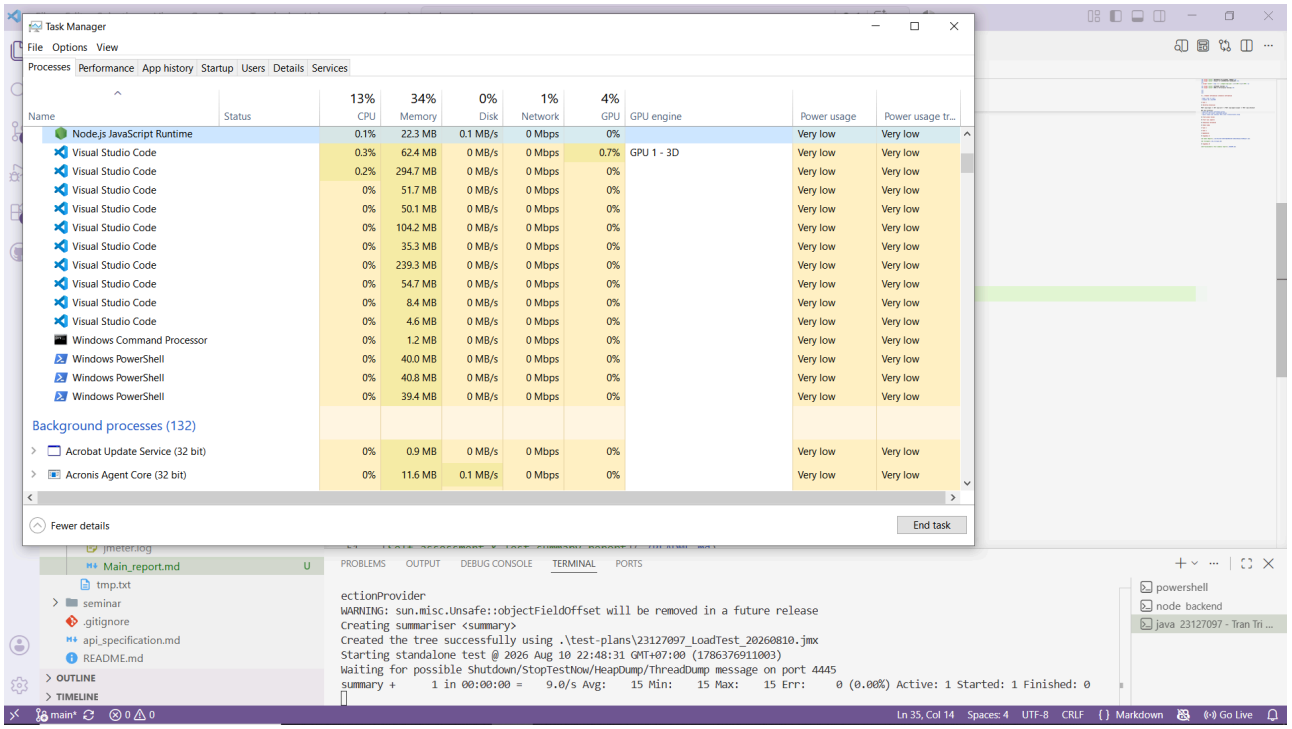
- In the Stress test plan, the AI suggested using `"user_id": "${__P(user_id,)}"` in the body of apply-coupon request, which would take the parameter at run time from the CLI. Instead, I chose to use JSON extractor in the login request to find the user id because with that, user id can be automatically obtained without having to explicitly include it in the CLI command.
- In the Spike test plan, I added JSON extraction for the user id in login request and use it in the apply-coupon request instead of defaulting it to 1. I also added the final amount assertion in the apply-coupon request to ensure that it can be extracted later on

For the Listeners(report view), I did not strictly follow the AI's suggestions but instead I explicitly chose View Results Tree for Load test plan, Aggregate Report for Stress test and Summary Report for Spike test.

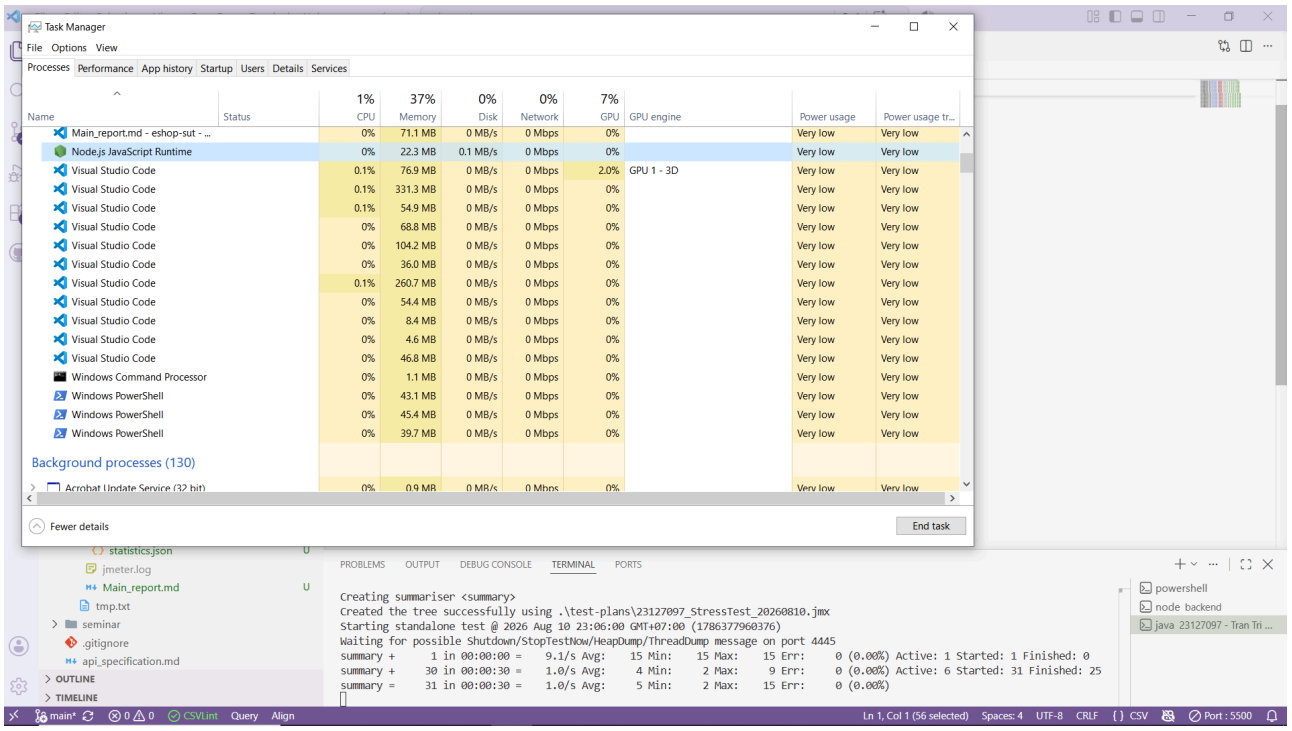
Test runs capture

Test results

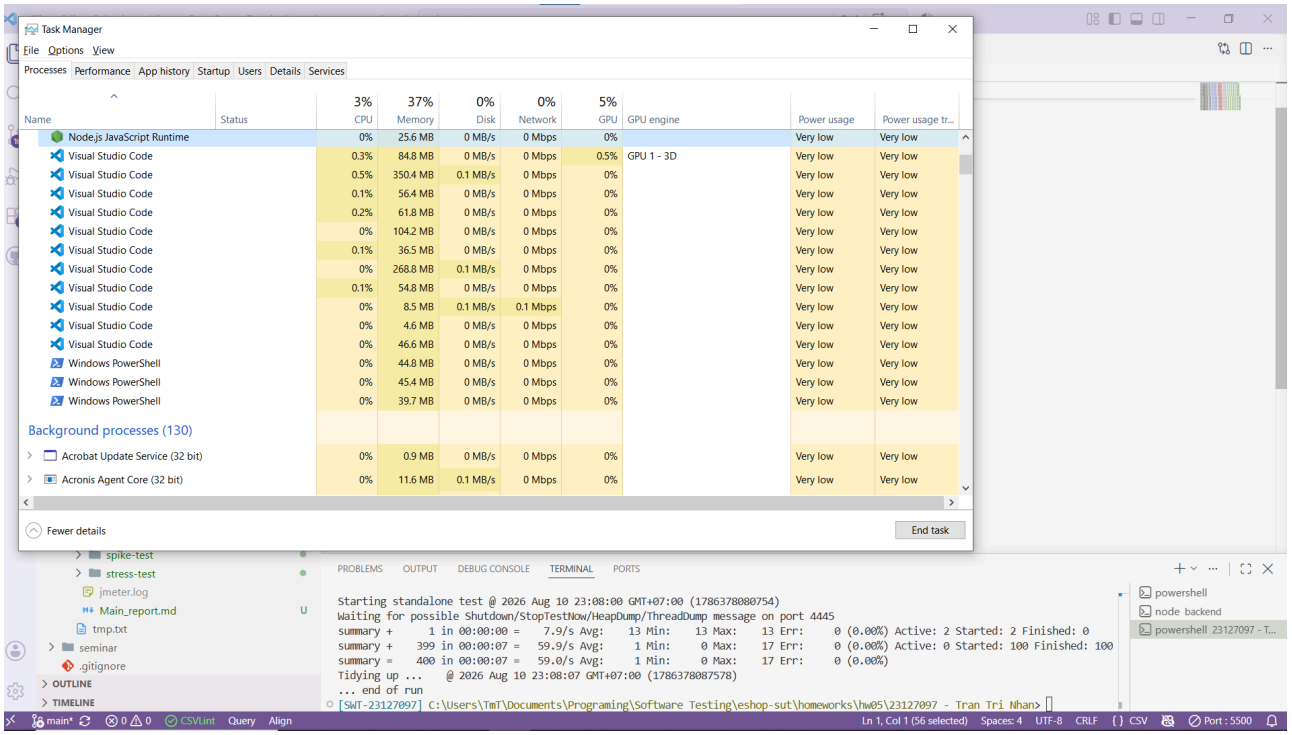
- Load Testing: PASS 100%
    - Screenshot:



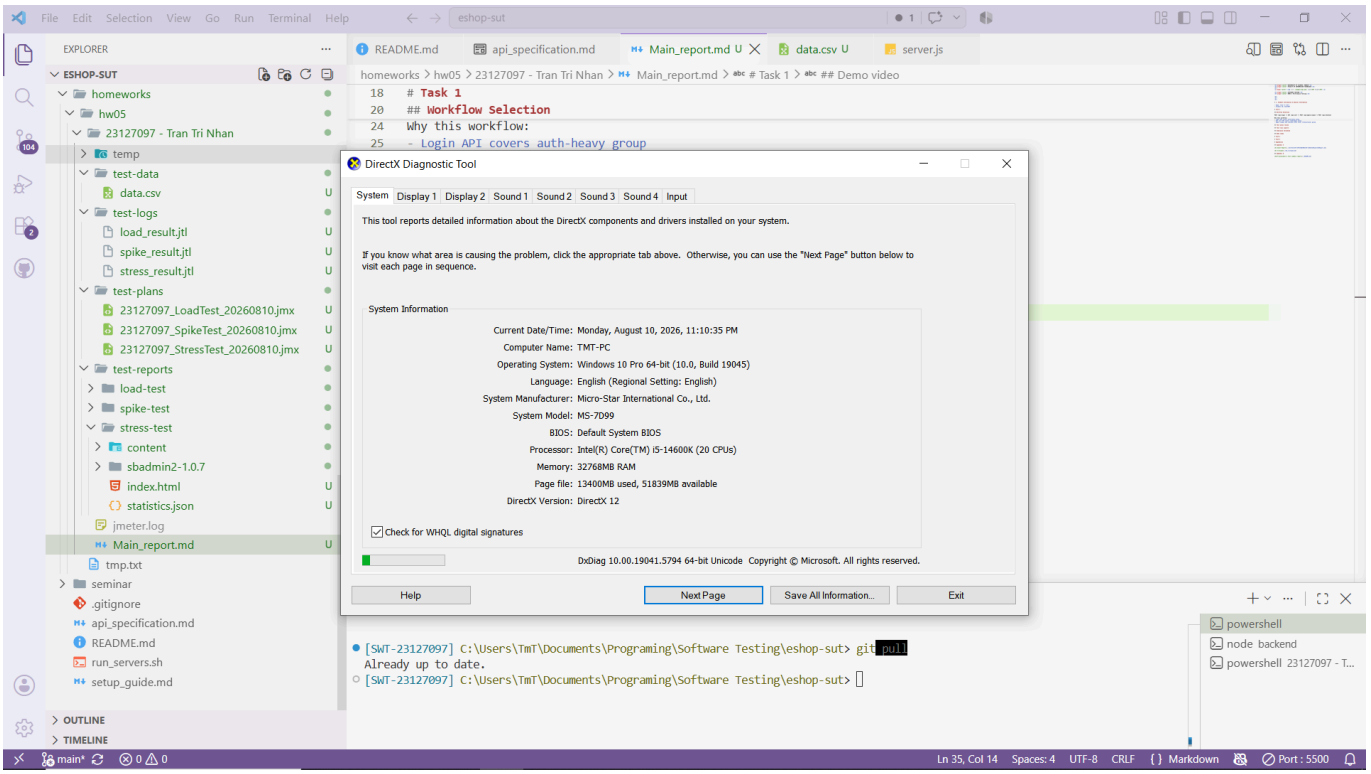
- Stress Testing: PASS 100%
    - Screenshot:



- Spike Testing: PASS 100%
  - Screenshot:



dxdiag capture



Spec table

Component	Specification
Operating System	Windows 10 Pro 64-bit (10.0, Build 19045)
Processor	Intel(R) Core(TM) i5-14600K (20 CPUs)
Memory	32768 MB RAM
RAM	32.0 GB
JMeter version	5.6.3
Java version	java 25 2025-09-16 LTS

Endurance test

- Settings:
  - Number of VUs: 100
  - Ramp-up duration: 60 seconds
  - Hold duration: 600 seconds
  - Targeted endpoint: GET /api/products
- Results:
  - Response Times(ms):
    - Average: 8.51
    - P50: 9.00
    - P90: 10.00
    - P95: 11.00
    - P99: 15.00
  - Throughput: 11162.84 transaction/s

- Peaked CPU Usage on backend: 11.4%

Demo video

Youtube link

Task 2

AI's analysis, suggested performance thresholds and optimization proposals

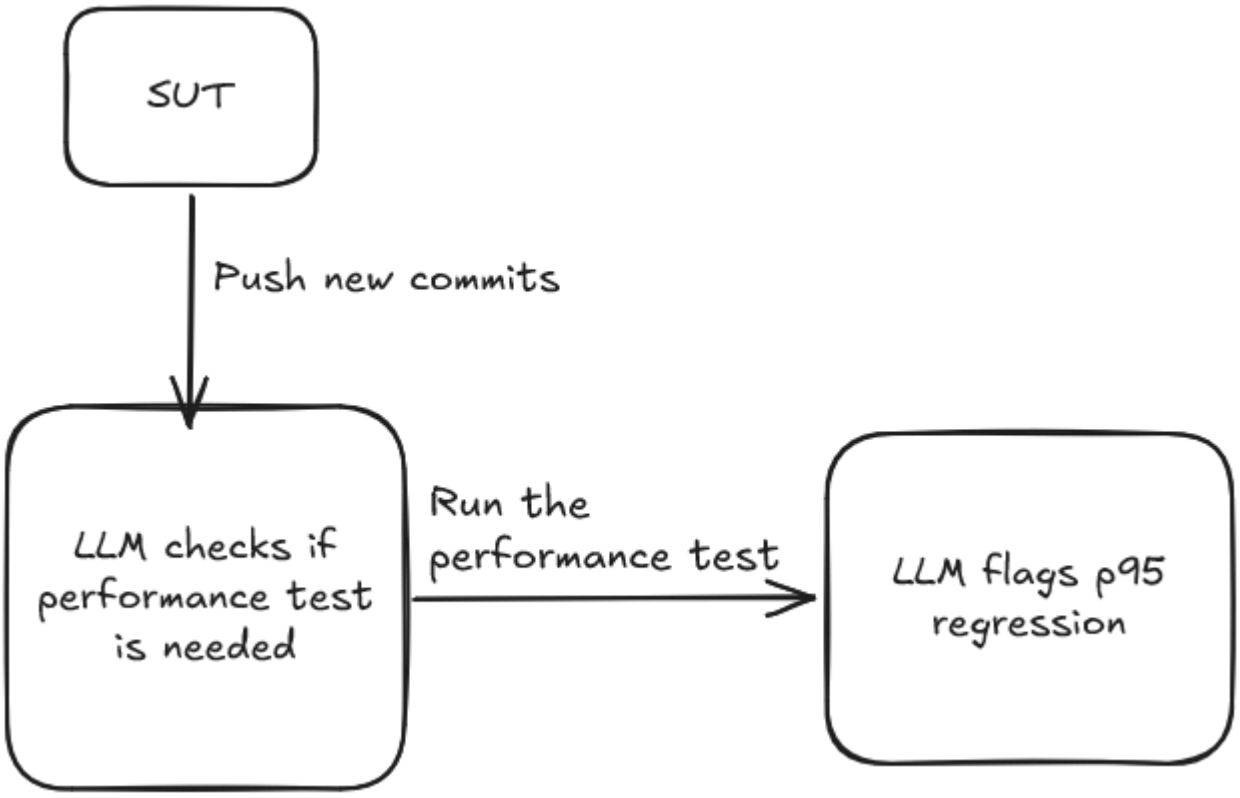
The AI correctly pointed out the concurrency vs latency correlation and the slowest endpoint. The suggested thresholds given by the AI are accurate and fully supported by the jtl logs. However, the AI suspected that the login request might be slow due to bcrypt hashing, which is incorrect because looking at the logs, the actual slowness came from the initial connection handshakes between the VUs and the server. For example, in the Load test jtl log, on the login request of thread Checkout_Flow_Users 1-1, it spent most of its time for connection(15ms total elasped and 10ms on connection), while its other requests were only around 1-9ms.

Optimization proposals review:

- SQLite WAL mode: feasible. Reason: Allow multiple read requests and stop them from be blocked while write operation is still going. Can be implemented easily by changing journal mode to WAL.
- Adding indexes: feasible. Reason: Can be applied to the coupon code column to speed up the response since the apply-coupon request includes this and is called in every transaction in the test plans.
- Adding a connection pool: hallucinated. Reason: Unfit for the server of EShop, because it uses single-thread instead multiple threads to handle requests.
- Re-run the load/stress/spike tests at meaningfully higher concurrency and longer duration: feasible. Reason: Numbers of Threads and Duration settings can be easily adjusted to rerun
- Caching: feasible. Reason: we can choose in-memory caching by using arrays in the backend code to store the products or categories, simple and easy to implement given the scope of the SUT.

Task 3

Flowchart of the continuous performance-testing model:



Trade-offs:

- Cost: This model could introduce token cost because it requires a Large Language Model acting as a judge to decide whether to run performance tests or not(and possibly which features we need to run the tests on) and to flag the possbile performance regressions after the run.
- False alarms: If the LLM gets hallucinated while deciding to run performance tests, it might run the tests on the simple features and miss out the critical, sensitive features that would actually require to be tested thoroughly. This further increase the development overhead because it now requires human reviewers to double check the decisions and the final run results.

Appendices

Appendix A

AI Audit Report

AI Critique

Appendix B

Self-assessment & Test summary report